

Toxic Comment Classification For E-Commerce Website

Ranjit Paswan, Rajat Bhar, Rishi Maharaj, Jyoti Nagle

B.TECH

in

Computer Science and Engineering

Under the Supervision of

Prof. Name : Sujata Dawn

Department of Computer Science and Engineering Durgapur Institute of Advanced Technology and Management Rajbandh, Durgapur

Abstract

Online communication has become prevalent within today's society. The issue with such platforms is that people are allowed to express what they want without repercussion. Consequently, toxicity on these platforms becomes common. One approach to limit such inappropriate messages could be using a filtering method. The thesis will discuss how to create a toxicity filter using machine learning along with an API for filtering messages by using the models created. The study also analyse which models perform the best in terms of three metrics: accuracy, precision and recall. The results indicate that KNN had the best result for predicting multiple variables while SVC and Logistic Regression worked best on single variable. Thus, making machine learning a viable method for filtering toxic messages. The increase in penetration of usage of internet services has increased exponentially in the past 4 months due to the ongoing pandemic, this has empowered an enormous number of dynamic new and old clients utilizing the web for different administrations ranging from academic, entertainment, industrial, monitoring and the emergence of a new trend in the corporate- life i.e work-from-home. Due to this sudden emergence of the crowd using the web, there has been an ascent in the number of mischievous persons too. Now it is the primary task of every online platform provider to keep the conversations constructive and inclusive. The best example can be referred to, can be twitter, a web-based media stage where people share their views. This platform has already drawn a lot of flak because of the spread of hate speech, insults, threat, defamatory acts which becomes a challenge for many such online providers in regulating them. Thus, there is active research being conducted in the field of Toxic comment classification. Here we collate non-identical machine learning and other trivial techniques on the dataset and propose a model that outflanks all others and compares them one-on-one. We have undertaken the Kaggle dataset for the above reason which has been broadly used and one of the prime resources for scholars working in deducing the challenge of toxic comment classification. The results would help up to create an online interface where we would be able to identify the toxicity level in the given phrase or sentence and classify them into their order of toxicity [1].

Keywords:

Machine Learning, Regression, Classifier, Toxicity Filter, API, Tensorflow, Scikitlearn, Binary relevance, Defamatory, Multinomial naive bayes, Support vector machinr, Toxic comment.

1 Introduction

Toxic comments are defined as comments that are rude, disrespectful, or that tend to force users to leave the discussion. If these toxic comment can be automatically identified, we could have safer discussions on various social networks, news portals, or online forums. Manual moderation of comments is costly, in- effective, and sometimes infeasible. Automatic or semi-automatic detection of toxic comment is done by using different machine learning methods, mostly different deep neural networks architectures. Recently, there is a significant number of research papers on the toxic com- ment classification problem, but, to date, there has not been a systematic literature review of this research theme, making it difficult to assess the matu- rity, trends and research gaps. In this work, our main aim was to overcome this by systematically listing, comparing and classifying the existing research on toxic comment classification to find promising research directions. The results of this systematic literature review are beneficial for researchers and natural language processing practitioners. Lately there has been many cases in which the growing menace of hate and negativity has been witnessed in the online platforms especially social media as such, many governments around the world has seen the rise of cases related to cyber bullying that has led to spread of hatred and violence. Since the democratization of substance creation following the dispatch of web-based media stages, every single one of us has become content makers making and distributing our own substance, which thus has made a framework where the nature of distributed substance cannot, at this point be controlled. The effect of the most recent twenty years' innovation unrest is presently affecting organizations, political frameworks, family lives, society, and individuals [1]. Detecting Toxic comments has been a great challenge for the all the scholars in the field of research and development. This domain has drawn lot of interests not just because of the spread of hate but also people refraining people from participating in online forums which diversely affects for all the creators/content-providers to provide a relief to engage in a healthy public interaction which can be accessed by public without any hesitation. There have been sure turns of developments in this area which includes couple of models served through API. But the models still make errors and still fail to provide an accurate solution to the problem. In this paper we have widely discussed a set of models which is utilized for text classification. These models/methods have been widely used in various fields such as economics, medical and environmental studies [2].

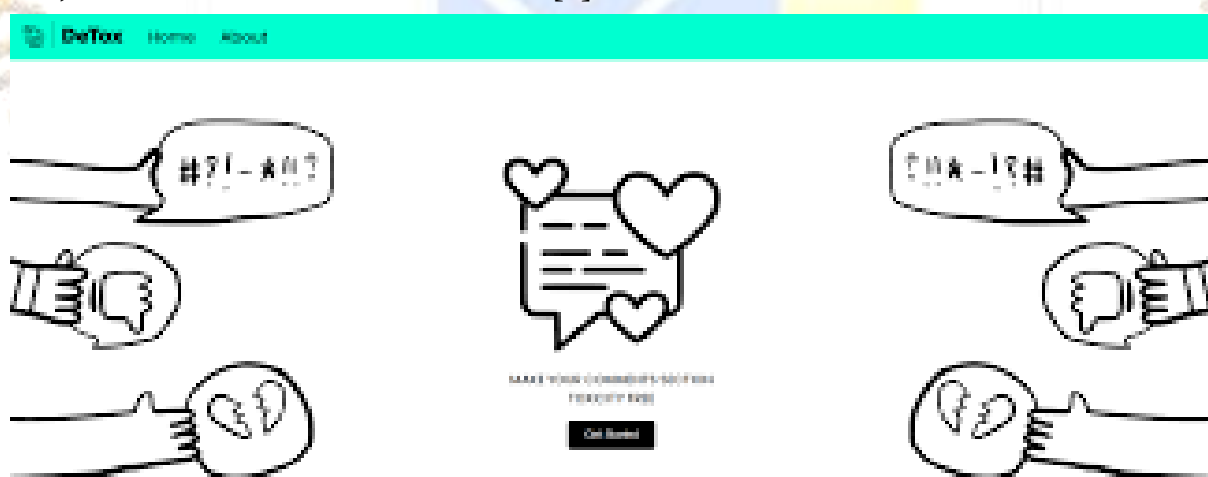


Figure 1: Introduction of Toxic Comment.

2 Motivation

The motivation behind developing a toxic comment classification model for an e-commerce website stems from several key factors:

1. **User Experience Enhancement** : Providing a safe and respectful environment for users is crucial for fostering a positive user experience. Filtering out toxic comments ensures a more enjoyable interaction for visitors and encourages engagement within the e-commerce platform.
2. **Brand Reputation and Trust** : Toxic comment or offensive content can damage the reputation of e-commerce platform. By swiftly identifying and handling such comments, the platform can maintain a trustworthy image and brand integrity.
3. **Community Guidelines Enforcement** : Upholding community guidelines and standard is essential for creating a healthy and respectful community within the e-commerce platform. Automating the detection of toxic comments assists in enforcing these guidelines consistently.
4. **Moderation Efficiency** : Automating the initial identification of toxic comments reduces the workload on human moderators, allowing them to focus on more nuanced or complex cases and ensuring a faster response to potential issues.
5. **Risk Mitigation** : Preventing harmful or offensive content from spreading can mitigate legal risks and prevent potential negative impacts on users mental well-being, thus creating a safer online space [3].

3 Project Objectives

- **Model Development** : Create an efficient classification system to accurately identify toxic comments. And creating a robust machine learning or natural language processing model capable of accurately identifying and classifying toxic comments on the e-commerce platform.
- **Integration** : Seamlessly incorporate the model into the e-commerce platform for real-time monitoring and handling of toxic comments.
- **Scalability and Adaptability** : Ensure the system can evolve to address new forms of toxic language. Designing a system that can scale with the platform's growth and can adapt to new types of toxic language or comments that might emerge.
- **Compliance and Monitoring** : Upload community guidelines and continuously monitor and improve the model's performance. And collect user feedback to improve its accuracy and effectiveness over time.
- **Accuracy and Efficiency** : Developing a model that efficiently filters and flags toxic comments while minimizing false positives and false negatives, thus enhancing the overall accuracy of classification.
- **Community Guidelines Enforcement** : Ensuring that the model's classification aligns with the helps enforce the platform's community guidelines and standards.
- **Legal and Compliance consideration** : Addressing legal and compliance aspects related to content moderation and user-generated content within the platform [4].

4 Literature Review

Toxic comments on social media platforms have been a source of a great stir between individuals and groups. A toxic comment is not only verbal violence but includes the comment that is rude, disrespectful, negative online behaviour, or other similar attitudes that make someone leave a discussion. Therefore, the toxic comments identification on social platforms is an important task that can help to maintain its interruption and hatred-free operations. Consequently, a large variety of toxic comment approaches have been proposed. Three characteristics concerning toxic classification are evaluated: classification, feature dimension reduction, and feature importance [2].

- “Impact of SMOTE on Imbalanced Text Features for Toxic Comments Classification using RVVC Model”** : The Author of this book discusses about the ensemble approach called regression vector voting classifier(RVVC), to identify the toxic comments over the social media platforms. The data-set for this is taken from Kaggle which is a multi-label data-set which contains labels as toxic, severe toxic, threat, insult, and identity hate. The values of the data set are given as binary values which contains 158,640 comments in total with toxic comments. Owing to the study of higher accuracy ensemble model their experiment indicated the good performance and they combined the LR and SVC model to get the higher accuracy which is known as RVVC. They conducted the experiments using originally imbalanced data-set with TF-IDF and BoW separately. The results proved that RVVC outperforms all other individual models when TF-IDF features are used with SMOTE balanced dataset and achieves an accuracy of 0.97. Despite the better performance of proposed model, its computational complexity is higher than individual models [5].
- “Toxic Comment Classification Implementing CNN combining Word Embedding Technique”** : The author of this says that Despite this model, it only classifies into six labels which further improves to another model where it first classifies whether the comment is positive or negative and then classifies into six labels. The dataset used is Wikipedia’s talk page edits, collected from Kaggle. They proposed the ensemble model called Convolution Neural Network(CNN) which is structured as data cleaning, adopting the NLP techniques, stemming, and converted word into vector by word embedding techniques. The accuracy for this model is calculated based on ROC – AUC. The receiver operating characteristic curve (ROC) score is 98.46percent and the area under score (AUC) score for this model is 98.05percent which is much accurate than the previous model and the existing works. There’s another technique where it uses the deep learning model recurrent neural networks(RNN) to perform text classification on a multilabel text dataset to identify different forms of internettoxicite [6].
- “Application of Recurrent Neural Networks in Toxic Comment Classification”** : This paper discuss about this aspect. The dataset used for this methodology is the public dataset which is provided by Conversation AI team. Even Though the previous models has given the accurate toxicity scores, the models still miss classify some texts that share similar patterns as toxic comments which can be reduced using the RNN method. They used the Word 2 Vec embedding model to train the model to remove the noise in the data and the methodologies used are the recurrent neural network and done the comparison analysis with GRU. They have successfully employed the word2vec embedding and recurrent neural network in building a toxic comment classification model and achieved high accuracy with low cost. Even with the existing models have achieved high accuracy, building the model takes more time and complexity of the model will higher. This leads to Automatic detection [7].

- **“Automatic toxic Comment Detection Using DNN”** : This paper discusses about the automatic tools which were build using the LSTM and RNN to improve the accuracy and provide the results much faster than the traditional methods. This paper compared three state-of-art unsupervised word embedding models which were Mikolov’s word embedding, fastText subword embedding, BERT Wordpiece model. The dataset used is Wikipedia Detox Corpus which talks about English Wikipedia talk pages. All the models were compared against the BERT fine-tuning. And on experiments and results, it showed that BERT fine-tuning is the most efficient model at this automatic toxic classification task. This can further developed into the speech toxic recognition using more advanced models like XLNet model. There’s another model where the hybrid models are produced to get the accurate score much more than CNN model [8].
- **“Toxic Comment Classification Using Hybrid Deep Learning Model”** : In this paper discusses about the hybrid models used which are Bidirectional gated recurrent network, convolution neural network and they achieved the accuracy of 98.39 and the f1 score of this model was 79.91 percent. As much as toxic comment classification is important, toxic span prediction also play the similar role which helps to build more automated moderation systems [9].
- **“Multi-Task Learning For Toxic Comment Classification And Rationale extraction”** : The Paper discuss about the multi-task learning model using the Toxic XLMR for bidirectional contextual embeddings of input text for toxic comment classification and a Bi-LSTM CRF layer for toxic span and rationale identification. The dataset used was curated from Jigsaw and Toxic span prediction dataset. The model has outperformed the single task models on the curated and toxic span prediction models by 4 percent and 2 percent improvement for classification. The future improvements can be added to take more delicate context and handle the subtle differences in usage of keywords [10].
- **“Vulgarity Classification In Comments Using SVM and LSTM”** : This Paper discusses about the hybrid model of SVM and RNN-LSTM. The data is vectorized using TF-IDF and bag of words. This paper also discusses the nature of the dataset. The results found to give a promising assurance in finding a solution [11].
- **“Modern Approaches to Detecting and Classifying Toxic Comments Using Neural Networks”** : In This Paper discusses about the algorithms constructed using deep learning technologies and neural networks that solve the problem of detecting and classifying toxic comments. The algorithms are tested and trained on a large training set and tagged by Google and Jigsaw which was taken from Kaggle [12].

5 Methodology

This section will go through the methods used to develop, analyze and extract data for the models. The methodology consist of a literature study where previous studies are analyzed for their methods used. The literature study is then used for the resulting evaluation of what tools should be used for the implementation. Every tool, framework and library used in this thesis is free (at least for educational purposes). This study uses different techniques, methods, and tools for the classification of toxic and non-toxic comments. Also, various preprocessing steps, data re-sampling methods, features extraction techniques, and supervised machine learning models are adopted for the said task.

5.1 Tools, Frameworks and Libraries

This section will discuss the tools, frameworks and libraries that were used and how they were used in this project.

- **Google colab** : Google Colab is an online platform provided by Google that allows users to write, execute, and share Python code in a Jupyter Notebook environment. It provides free access to GPU and TPU resources, making it ideal for running deep learning experiments and training models efficiently. In this project, Google Colab serves as the development environment where the code for training the machine learning model is written and executed.
- **Python** : Python is a high-level programming language known for its simplicity and readability. It's widely used in various domains, including machine learning and data science, due to its extensive libraries and frameworks. In this project, Python is the primary language used for writing the code to train and evaluate the machine learning model.
- **PyTorch** : PyTorch is an open-source machine learning framework developed by Facebook's AI Research lab. It provides a flexible and intuitive platform for building and training deep neural networks. PyTorch's dynamic computational graph feature allows for dynamic and efficient model building. In this project, PyTorch is used for implementing the neural network architecture and training the BERT model for text classification.
- **Transformers Library** : The Transformers library, developed by Hugging Face, is an open-source library built on top of PyTorch and TensorFlow for working with transformer-based models in natural language processing (NLP). It provides pre-trained models, tokenizers, and utilities for fine-tuning and inference tasks. In this project, the Transformers library is used for accessing the pre-trained BERT model and tokenizer for text classification.
- **Pandas** : Pandas is a data analytics tool for python, often used when working on ML for reading, parsing, and filtering the data. Within ML it is primarily used to read in the data into the RAM and remove invalid data. Large file can be read in chunks. Combining this with a model that can train multiple times, a 1 TB file can be trained with only 16 GB of RAM. Data cleaning in Pandas can be done in different ways depending on the data that needs to be cleaned. For example, timestamps are challenging to sort out because they can be in many different formats (year, month, day, hour, minute, second and millisecond) as well as any order and written with/without beginning zero or century. On top of this, each entry does not need to be on the same timezone or coded in the same way. Whether or not the clock is timestamped as AM/PM is impossible to know before hand on every dataset. More general data cleaning in pandas can for example be remove all null values with a function or remove all rows where a value is greater, smaller or equal to a number. Pandas was the primary library for manipulating the data used in this project. It is a python library and a framework, using a notebook and working with Pandas as a tool was useful for the data cleaning in this project [13].
- **NumPy** : NumPy is a fundamental package for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently. In this project, NumPy is used for numerical computations and data manipulation tasks.

- **Matplotlib and Seaborn** : Matplotlib and Seaborn are data visualization libraries in Python. They provide a wide range of plotting functions and customization options for creating various types of plots and charts to visualize data. In this project, Matplotlib and Seaborn are used for visualizing the distribution of toxic and clean comments, as well as for other data exploration tasks.
- **Scikit-learn** : Scikit-learn (Sklearn) is a library for data science such as ML. It works by providing different models for training on a dataset provided by the user and supports both regression and classification among other models. Sklearn was used for example in a thesis for the Logistic Regression and the Random Forrest models. This library was used in this project for creating every model except the CNN because CNN is a deep neural network. The syntax for each model is the same for predicting and fitting the model which makes it very easy to swap out models and test different techniques. The reason Sklearn was used in this report was primarily because it had an easy syntax and good results in previous studies [14].
- **Google Drive** : Google Drive is a file storage and synchronization service provided by Google. It allows users to store files in the cloud, synchronize files across devices, and share files with others. In this project, Google Drive is used for storing and accessing the dataset containing comments labeled with toxicity categories, as well as for storing and accessing the trained model files. It's integrated with Google Colab, enabling seamless access to files stored in Google Drive from within the Colab environment.

5.2 DataSets

The datasets that were used to make the analysis were found on Kaggle for free after creating an account. The dataset included 159571 entries. The ratio between training data and testing data was 3:1, meaning 25 of the data was used for testing and calculating the metrics of each model. Kaggle has a wide variety of toxicity challenges where the datasets provided for those challenges was extremely useful. There were other datasets found on github etc. They were however too small. Combining them could be an option but Kaggle had already a large dataset [15].

5.3 Preprocessing

ML has a fixed number of inputs for each trained model. It is impossible to convert the text to corresponding ASCII value and feed the model, simply because it is not a guarantee every message will be of equal length. One of the solutions to this problem is to use a library called Spacy. A thesis about NLP explains the problems about processing text and ways of preprocess text in chapter 3. Spacy has a tool for taking a text as input and output a 300 dimensional vector. This solves the problem of converting text to numbers described earlier. The downsides of using Spacy is language dependency, it is required to know the language beforehand. Therefore the restrictions to the models will only use English as input [16].

5.4 Models

To begin with the analysis, a comparison of accuracy between the models was made. Some of the models were: KNN, CNN, Logistic Regression, and SVC. These models were used in previous works, additionally KNN and CNN were also used in a course on applied machine learning for a classification problem at KTH and therefore were used in this thesis. One side note to some of the models is the fact that the regression models return an estimation of how sure it is of a message being toxic. This is bad when a service uses the model and requests a simple yes/no. Since regression models does not have to be contained between 0 and 1, if the output becomes -0.3 does that mean its a kind sentence or the model is unsure there is an error. In this case a threshold of 0.5 could determine if the message is toxic or non-toxic but why not just use a classifier in that case if you turn a regressor into a classifier [17].

5.5 Training

The training of most of the models took less than 5 minutes for above 90percent accuracy. The training was done using Jupyter Notebook. The computers that were used for training of the models were laptops with these specs [18]:

1. CPU: Intel Core i5 8th gen. 8 GHz.
2. RAM: 8 GB.
3. Operating System: Windows 10 home.

5.6 Implementation

This section will describe how to create a ML model for toxic messages. To start code the models, prerequisites and libraries need to be installed. The following is a list of each library and framework that were used for implementing the models. VS Code is not required for this project, any other IDE with similar properties would work, but because of the familiarity VS Code was used. Similarly any dataset with similar properties would work but because Kaggle was used by similar reports it was also used here. Prerequisites include [19]:

1. Python 3.9 or later. (required)
2. Pip 21.3 or later. (required)
3. Dataset from kaggle (kaggle has several of the largest quality datasets on this topic). (required)
4. Visual Studio Code 1.66 or later. (optional)

With the prerequisites installed, the python libraries can be installed using pip. Similarly, Pandas can be used to do similar functionalities, for example Dask (it is more complicated but faster). Libraries like Spacy and Tensorflow are harder to replace. Libraries include:

1. Pandas 1.4.1 or later (numpy is a dependency and will be automatically installed).
2. Tensorflow 2.8.0 or later.
3. Scikit learn 1.0.2 or later.
4. Spacy 3.2.4 or later.

5.7 Data Cleaning

During data cleaning it is important to understand what is crucial for the sentence, for example "i think you are an idiot." could be replaced with "i think you idiot" for the same meaning but about half the size. The computer cannot understand what is important so it will assume everything is equally important, data cleaning makes sure that the input data only contains information that is crucially important for the model. [20].

5.8 Reading Cleaned Data

The problem with reading the cleaned data is that Pandas perceives the data as a string and not an array. To convert it from string to array the reshape function in numpy is used. Once the comment vectors are read as arrays and not strings, it can be used as a feature for the models. An alternative where this step is not required is if the vectors/arrays were split in two different columns. A problem there is the number of features change depending on the algorithm/library size in Spacy. Therefore when using the cleaned data the number of features needs to be known. The next step is to read the data as a stringed array and remove last and first characters (the brackets), flatten the string and fix so every number is separated by only one space. This way the only thing that is required is to split the string by spaces and an array of numbers is all that is left [21].

5.9 Modeling

Once the data is read and parsed from strings to arrays, the modeling can begin. Using only one Jupyter Notebook for all models violates clean code because they are independent but forced to use the same resources. It is better to let the models be completely independent because changing the testing model code etc., will only affect one model [22].

5.10 Transformer Architecture

BERT is based on the transformer architecture, which was introduced in the paper "Attention is All You Need" by Vaswani et al. Transformers use self-attention mechanisms to capture relationships between different words in a sequence, enabling the model to understand the context of each word based on its surrounding words.

5.11 Bidirectional Context

Unlike previous models that process text in a unidirectional manner (from left to right or right to left), BERT is bidirectional. It can consider the entire context of a word by processing both left and right context simultaneously. This bidirectional approach allows BERT to capture deeper semantic meaning and context from the input text.

5.12 Transfer Learning

After pre-training on large text corpora, BERT can be fine-tuned on downstream tasks with labeled data. Fine-tuning involves updating the parameters of the pre-trained BERT model using task-specific labeled data. This allows BERT to adapt its learned representations to perform well on specific NLP tasks such as text classification, named entity recognition, question answering, and more.

5.13 Tokenization

BERT uses WordPiece tokenization, which breaks down words into subword units. This allows BERT to handle out-of-vocabulary words and capture finer-grained linguistic information. BERT tokenizers encode input text into numerical representations suitable for processing by the model.

5.14 Save And Load The Model

Once the models are created in RAM, it is time to save them to the computer. This can be done using joblib and is easy to work with. To save any model using joblib, the dumps function will be used and to load any model. Because the API does not train a model everytime the server reboots it needs to load a model from memory.

5.15 Application Programming Interface

This section will describe how to create an API to use the models in a more practical way [23].

1. **Prerequisites** : Flask is a library and can be used to manage http requests. Flask can be installed in numerous ways, either via an IDE or the command line.
2. **Endpoints** : The API will allow the models to be used by the HTTP protocol and therefore be accessed via endpoints. The endpoints for the API are as follows. (a.) PUT /load. This endpoint will load the API with the model specified in the request. (b.) GET /filter. This endpoint will filter the message based on the current loaded model [24].

5.16 Analysis

This section will describe how the analysis of the models was done. Firstly, a model has to be generated either via Sklearn, keras or loaded from a file into a Google Colab. Secondly, Sklearn has a library for measuring models accuracy, precision and recall and can be used [25].

6 Results and Discussion

The accuracy, precision, and recall metrics obtained from evaluating the model on the testing dataset indicate how well the BERT-based classifier performs in identifying toxic comments across different categories. For instance, if the accuracy is high, it suggests that the model can accurately classify comments into their respective toxicity categories. Similarly, high precision indicates fewer false positives, and high recall suggests fewer false negatives. Visualizations, such as confusion matrices or precision-recall curves, can provide a more comprehensive understanding of the model's performance across different toxicity categories. These visualizations can highlight which categories the model excels in classifying and identify areas where it may struggle or produce misclassifications.

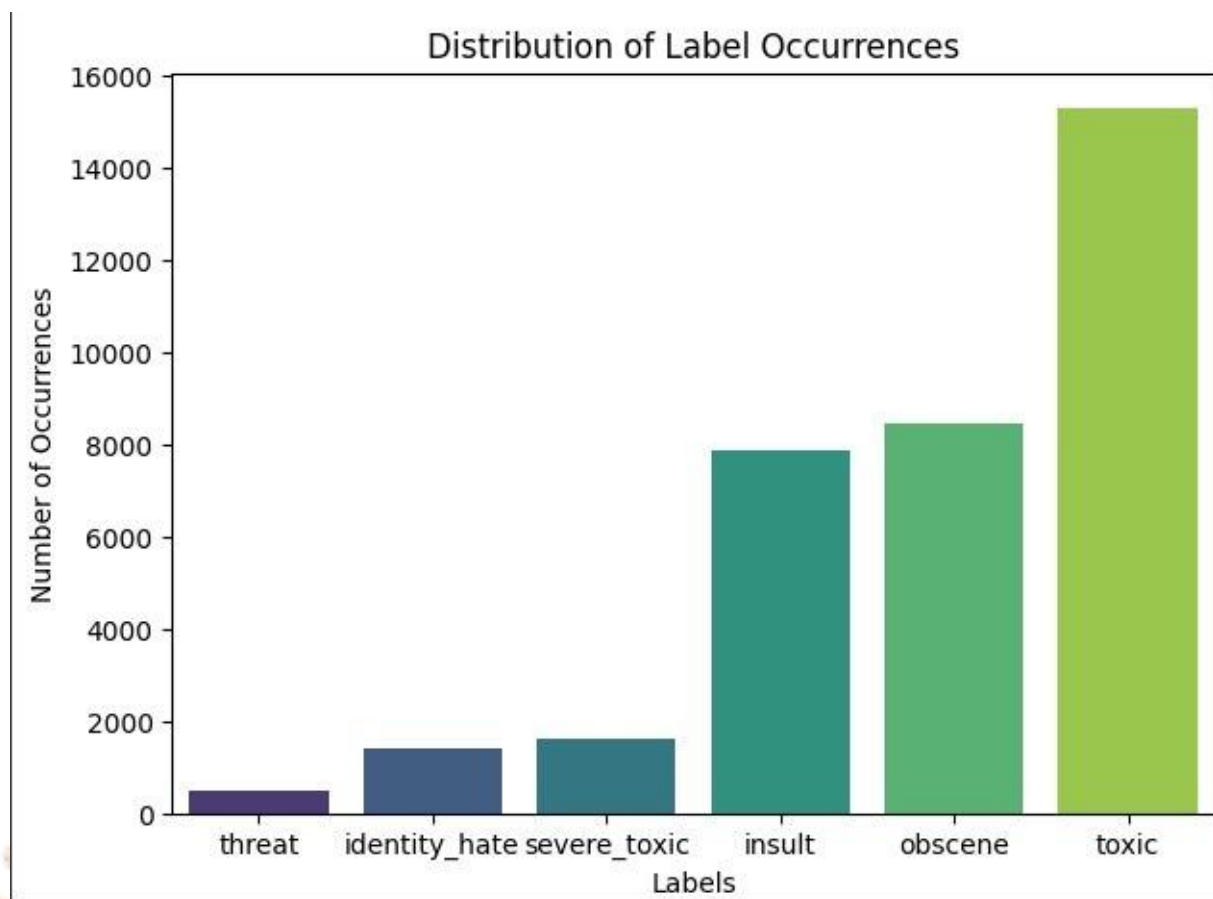


Figure 2: Distribution Of Label Occurrences.

BERT, being a state-of-the-art model, likely demonstrates strong performance in capturing nuanced linguistic features and context, leading to accurate toxicity classification. Its bidirectional nature allows it to consider the entire context of a comment, enabling more robust understanding and classification of toxic language. The dataset may exhibit class imbalance, where certain toxicity categories have significantly fewer examples compared to others. Addressing class imbalance through techniques like oversampling, undersampling, or class weighting could potentially improve the model's performance, especially for minority classes. Conducting error analysis by inspecting misclassified examples can provide insights into the model's weaknesses and areas for improvement. Identifying common patterns or recurring types of misclassifications can guide the refinement of the model architecture or training process.

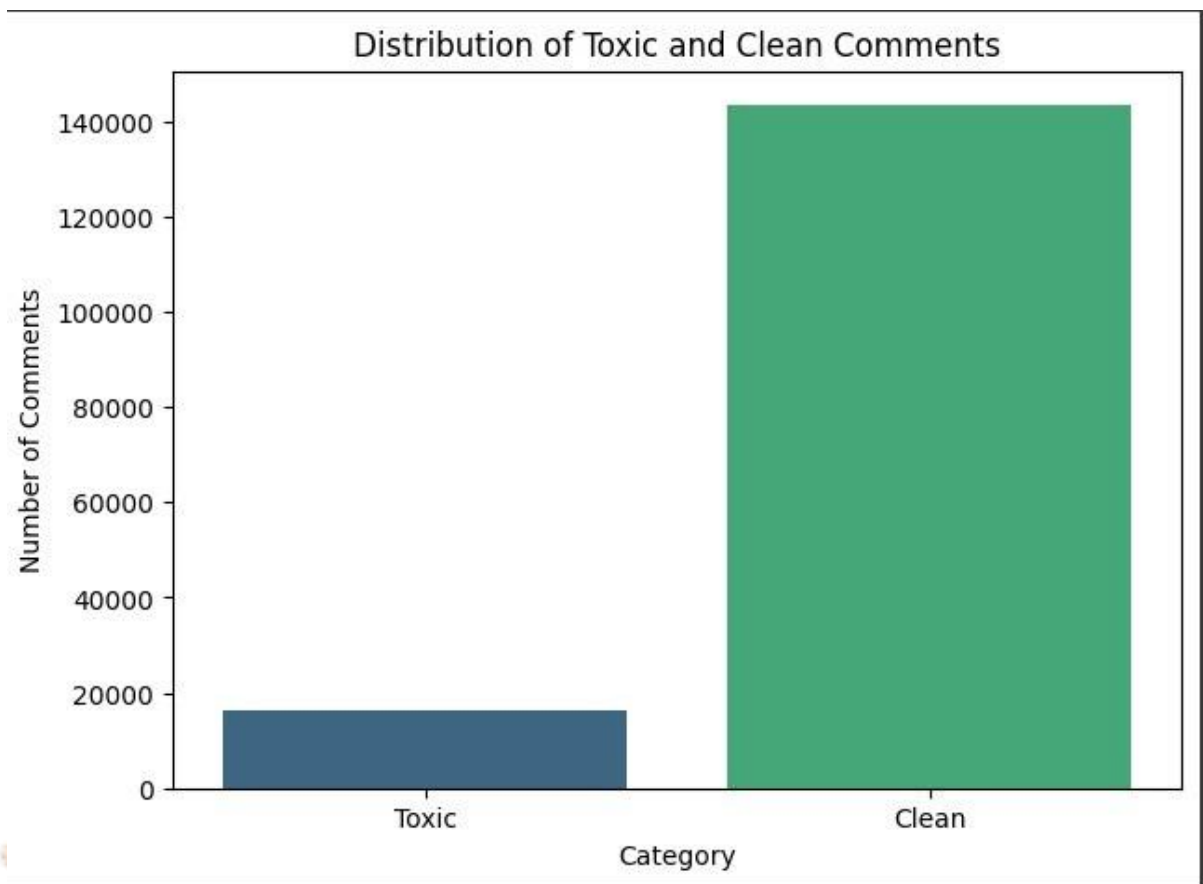


Figure 3: Distribution Of Toxic and Clean Comment.

To ensure high accuracy in Toxic Comment Classification, we employed a combination of BERT model, NLP and Tokenization. By using a mix of these methods, we aimed to get the most accurate predictions possible.

```
Epoch 1, Training Loss: 0.2134356197072548, Validation loss: 0.15295451803235557
Epoch 2, Training Loss: 0.13815239465914167, Validation loss: 0.1531994088780223
Epoch 3, Training Loss: 0.11428895100960985, Validation loss: 0.1529846265325396
```

Figure 4: Accuracy of used algorithms.

7 Conclusion

The focus of this thesis project is to create a ML model for filtering toxic messages, compare the models based on accuracy, precision and recall, then preserve the models so the API can use them. This makes the models more applicable and the possibility of swapping between models increases flexibility. The first step in solving this problem was to split the problems into smaller ones. The problem became finding a dataset with wide variety of toxicity, clean the data and convert it into a fixed length array to be used for the models. With the data, next step was to use it with different models and save models with high metrics. These models could then be used in the API for more platforms. The fact that ML can only deal with fixed size inputs was an interesting finding since it changed the data preparation to the extent that additional libraries had to convert the dataset to make it compatible with the model. Another major finding was uncovering models that could produce a good result such as SVC and Logistic Regression. The advantage of ML is that it is very good at finding patterns that are almost invisible. One example of this could be the ability to predict how long it will take for a taxi in New York to reach from one point to another based on New York taxi data. The main question for this thesis was whether ML has the ability to find a pattern in toxic sentences and normal ones. The disadvantage of ML is that it cannot

process different amounts of features, for example sentences of different lengths. To conclude, ML has the capability to be used for filtering toxic messages however, due to language dependency it is required that the models are used in an environment with similar messages to what it is trained to filter for. For example a community is always changing, new members join and old members leave, it is therefore recommendable to retrain the model(s) if the community changes a lot.

8 Future Work

Datasets has been the main focus when building these models. If a dataset contained large diversity of toxic comments, then it would make the models more accurate against more creative insults. A wide variety of toxic sentences in datasets for example "you are worth as much as a beetle" would make the model more versatile. Different languages would be required for developing multilingual models. Currently, only a few datasets exists in different languages (Turkish, Portuguese, Russian, French and Italian dataset and therefore it may not be possible to make a toxicity filter for every language. Additionally, if Spacy could support any language it would only put the limit on the datasets. Thus, as a future work, a suggestion would be to create more language support for Spacy and datasets with larger variety of toxicity, such as more creative insults which proved to be difficult to filter.

References

- [1] Sara Zaheri, Jeff Leath, and David Stroud. Toxic comment classification. *SMU Data Science Review*, 3(1):13, 2020.
- [2] Darko Androcec. Machine learning methods for toxic comment classification: a systematic review. *Acta Universitatis Sapientiae, Informatica*, 12(2):205–216, 2020.
- [3] Hao Li, Weiquan Mao, and Hanyuan Liu. Toxic comment detection and classification. In *CS299 Machine Learning*. Stanford University, 2019.
- [4] B Naseeba, Pothuri Hemanth Raga Sai, B Venkata Phani Karthik, Chengamma Chitteti, Katari Sai, and J Avanija. Toxic comment classification. In *International Conference on Hybrid Intelligent Systems*, pages 872–880. Springer, 2022.
- [5] Vaibhav Rupapara, Furqan Rustam, Hina Fatima Shahzad, Arif Mehmood, Imran Ashraf, and Gyu Sang Choi. Impact of smote on imbalanced text features for toxic comments classification using rvvc model. *IEEE Access*, 9:78621–78634, 2021.
- [6] Monirul Islam Pavel, Razia Razzak, Katha Sengupta, Md Dilshad Kabir Niloy, Munim Bin Muqith, and Siok Yee Tan. Toxic comment classification implementing cnn combining word embedding technique. In *Inventive Computation and Information Technologies: Proceedings of ICICIT 2020*, pages 897–909. Springer, 2021.
- [7] Siyuan Li. *Application of recurrent neural networks in toxic comment classification*. PhD thesis, UCLA, 2018.
- [8] Ashwin Geet d'Sa, Irina Illina, and Dominique Fohr. Towards non-toxic landscapes: Automatic toxic comment detection using dnn. *arXiv preprint arXiv:1911.08395*, 2019.
- [9] Rohit Beniwal and Archana Maurya. Toxic comment classification using hybrid deep learning model. In *Sustainable Communication Networks and Application: Proceedings of ICSCN 2020*, pages 461–473. Springer, 2021.
- [10] Kiran Babu Nelatoori and Hima Bindu Kommanti. Multi-task learning for toxic comment classification and rationale extraction. *Journal of Intelligent Information Systems*, 60(2):495–519, 2023.

- [11] Crystal Dias and Mahesh Jangid. Vulgarity classification in comments using svm and lstm. In *Smart Systems and IoT: Innovations in Computing: Proceeding of SSIC 2019*, pages 543–553. Springer, 2020.
- [12] Sergey V Morzhov. Modern approaches to detecting and classifying toxic comments using neural networks. *Automatic Control and Computer Sciences*, 55(7):607–616, 2021.
- [13] Jigsaw Multilingual. Jigsaw multilingual toxic comment classification, 2020.
- [14] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.
- [15] Joaquin Quinonero-Candela, Masashi Sugiyama, Anton Schwaighofer, and Neil D Lawrence. *Dataset shift in machine learning*. Mit Press, 2008.
- [16] Michal R Chmielewski and Jerzy W Grzymala-Busse. Global discretization of continuous attributes as preprocessing for machine learning. *International journal of approximate reasoning*, 15(4):319–331, 1996.
- [17] Revati Sharma and Meetkumar Patel. Toxic comment classification using neural networks and machine learning. *Int. Adv. Res. J. Sci. Eng. Technol*, 5(9), 2018.
- [18] Batta Mahesh. Machine learning algorithms-a review. *International Journal of Science and Research (IJSR)*.*[Internet]*, 9(1):381–386, 2020.
- [19] Aaron E Maxwell, Timothy A Warner, and Fang Fang. Implementation of machine- learning classification in remote sensing: An applied review. *International journal of remote sensing*, 39(9):2784–2817, 2018.
- [20] Xu Chu, Ihab F Ilyas, Sanjay Krishnan, and Jiannan Wang. Data cleaning: Overview and emerging challenges. In *Proceedings of the 2016 international conference on management of data*, pages 2201–2206, 2016.
- [21] Ihab F Ilyas and Theodoros Rekatsinas. Machine learning and data cleaning: Which serves the other? *ACM Journal of Data and Information Quality (JDIQ)*, 14(3):1–11, 2022.
- [22] Jared Willard, Xiaowei Jia, Shaoming Xu, Michael Steinbach, and Vipin Kumar. Integrating physics-based modeling with machine learning: A survey. *arXiv preprint arXiv:2003.04919*, 1(1):1–34, 2020.
- [23] Paulo Lissa, Conor Deane, Michael Schukat, Federico Seri, Marcus Keane, and Enda Barrett. Deep reinforcement learning for home energy management system control. *Energy and AI*, 3:100043, 2021.
- [24] Fabio Gasparetti, Carlo De Medio, Carla Limongelli, Filippo Sciarrone, and Marco Temperini. Prerequisites between learning objects: Automatic extraction based on a machine learning approach. *Telematics and Informatics*, 35(3):595–610, 2018.
- [25] Ivan Firdausi, Alva Erwin, Anto Satriyo Nugroho, et al. Analysis of machine learning techniques used in behavior-based malware detection. In *2010 second international conference on advances in computing, control, and telecommunication technologies*, pages 201–203. IEEE, 2010.